Accelerating Market Efficiency: A Heterogeneous High-Frequency Trading Engine for Tadawul

AUTHOR

Kamel Gerado — KFUPM ID: 202408240

SUPERVISOR

Dr. Mohammed Elrabaa

COURSE

Project (HPCC 619)

PROGRAM

MX in High Performance and Cloud Computing

King Fahd University of Petroleum & Minerals — May 2026

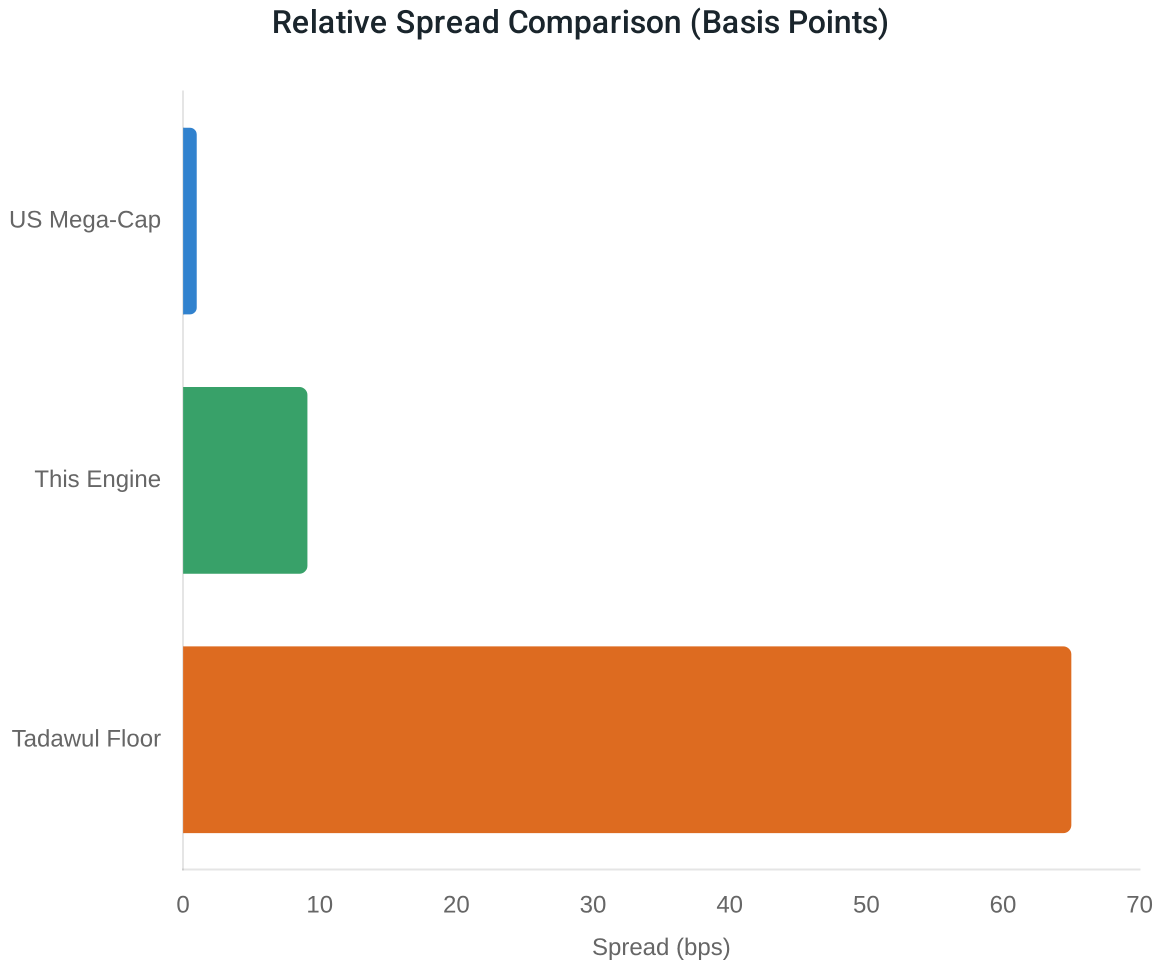
The Problem in One Number

> *"Tadawul requires market makers to quote within 65 basis points. US markets trade at 1 bp. That gap costs investors millions daily."*

Tadawul is the largest stock exchange in MENA, with a SAR 10 trillion market cap.

Group A market makers face a 65 bps spread floor—up to 650× wider than US mega-caps.

Vision 2030 aims for a top-10 global exchange, necessitating HFT-grade infrastructure.



This Engine: 7× tighter than regulatory floor

The Data: NASDAQ ITCH 5.0 and the 15-Symbol Selection

> *"No Tadawul data is publicly available — we use the closest structural equivalent: the top 15 NASDAQ stocks by market cap."*

Dataset: NASDAQ ITCH 5.0

- Binary record of every order book event.
- Jan 30, 2020 (Normal market conditions).
- 423.3M messages, 1.8GB compressed.

Structural Equivalence

Comparison to the 65 bps floor is **structural** (same liquidity tier), not a direct replay. Selected symbols satisfy Tadawul's Group A criteria.

NASDAQ Symbol	Market Cap (2020)	Tadawul Analogue
AAPL	~$1.4T	Saudi Aramco (2222)
MSFT	~$1.3T	SABIC (2010)
AMZN	~$940B	ALRAJHI (1120)
GOOGL	~$990B	SNB (1180)
FB	~$620B	RASAN (8313)
+ 10 more	...	...

WAMID DataHub: Access attempted 2024, not available for academic research.

Formal Problem Statement

> *"Four problems defined before building — each maps directly to a results section."*

P1 — THROUGHPUT & LATENCY

Build a CPU order-book engine processing ITCH 5.0 at ≥10 M msgs/sec with p99 latency < 1 µs per order.

P2 — MARKET EFFICIENCY

Implement a strategy that reduces spreads vs baseline and achieves spreads below Tadawul's 65 bps regulatory floor.

This Engine

P3 — GPU ACCELERATION

Design CUDA kernels for portfolio risk and analytics that achieve measurable speedup for 230+ symbols.

P4 — MICROSTRUCTURE

Implement all four standard spread measures and perform the Huang-Stoll adverse selection decomposition.

What is Market Making?

> *"A market maker continuously posts buy and sell prices, earning the spread."*

Bid and Ask

Simultaneous posting of buy (bid) and sell (ask) prices to provide liquidity.

The Spread

Profit is earned from the difference between the ask and bid prices per round trip.

Adverse Selection

The risk of price moving against a position before quotes can be updated or cancelled.

Order Book Example	
Ask Price	$215.65
Spread = $0.02	
Bid Price	$215.63

Scenario: Incoming buyer fills ask → market maker earns $0.02. If price moves UP immediately after, the market maker faces adverse selection.

Why GPU? The Scaling Problem

> *"Checking risk across 230+ symbols serially takes O(N) time. GPU reduces it to O(1)."*

CPU: Sequential Processing

[Symbol 1] → [Symbol 2] → [...] → [Symbol N]
Total Latency = N × t

- Checks each symbol one by one.
- Latency grows linearly with market size.
- Becomes a bottleneck for Tadawul's 230+ securities.

GPU: Parallel Processing

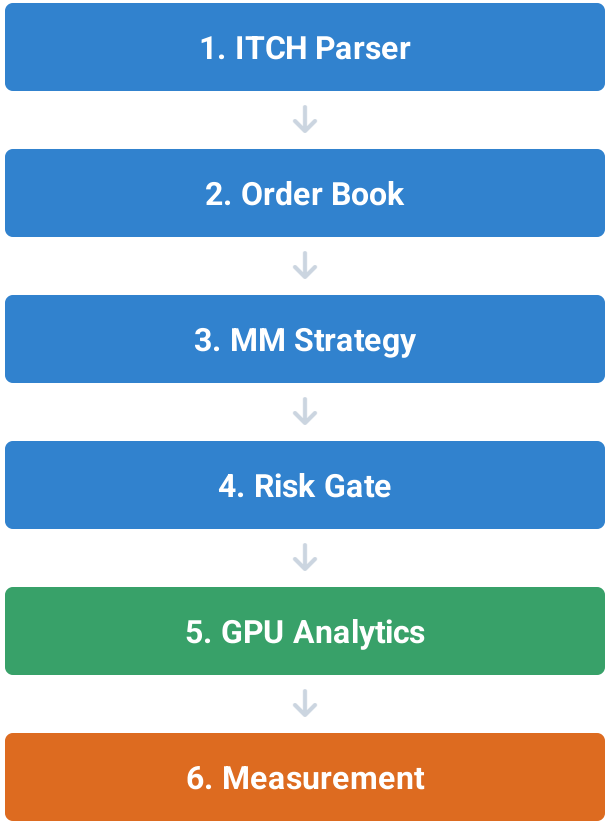
[Symbol 1]
[Symbol 2] ⇒ All N symbols at once
[Symbol N]
Total Latency = t (Constant)

- All N symbols checked in parallel.
- Constant time regardless of symbol count.
- Enables massive scaling for complex risk rules.

```
// GPU Parallel Reduction Concept
__global__ void risk_reduction(float* positions, float* result, int N) {
    __shared__ float partial[256];
    int tid = threadIdx.x;
    partial[tid] = (tid < N) ? fabsf(positions[tid]) : 0.0f;
    __syncthreads();
    for (int s = blockDim.x/2; s > 0; s >>= 1) {
        if (tid < s) partial[tid] += partial[tid + s];
        __syncthreads();
    }
```

System Architecture Overview

> *"6-layer pipeline: CPU handles events, GPU handles scale."*

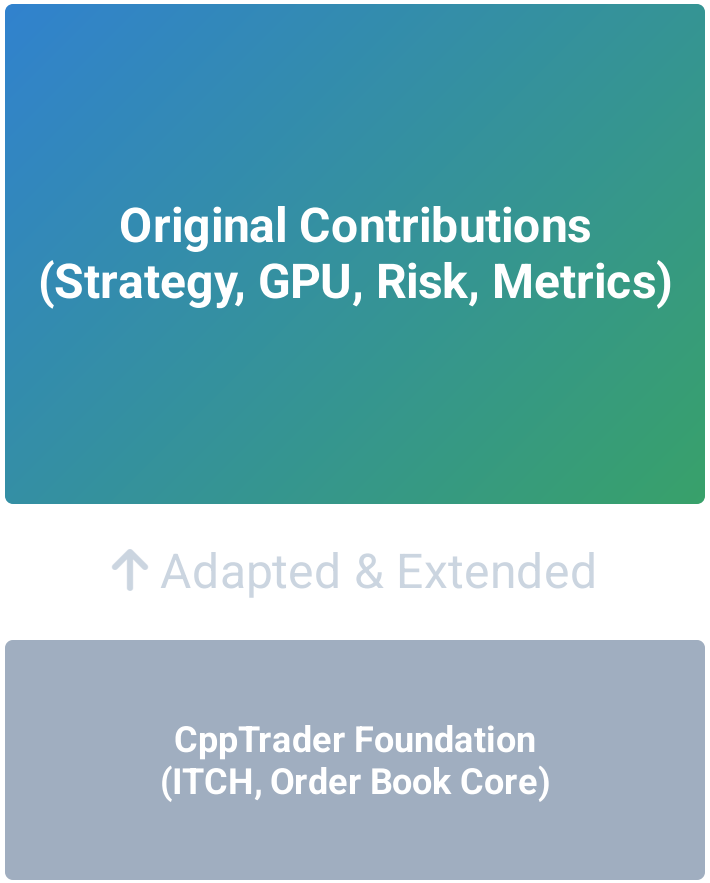


Layer	Function	Execution
1	ITCH Parser — reads market data	CPU
2	Order Book — maintains bid/ask levels	CPU
3	MM Strategy — generates quotes	CPU
4	Pre-Trade Risk Gate — checks exposure	CPU + GPU
5	GPU Analytics — volatility & scoring	GPU
6	Measurement — records snapshots	ASYNC

Latency-critical path stays on CPU; bulk parallel work is offloaded to GPU.

Original Contribution vs Adapted Foundation

> *"Built on CppTrader's order-matching core. GPU, strategy, risk, and measurement are 100% original."*



Adapted Foundation (Layers 1–2)

- ✓ ITCH 5.0 binary protocol handler (`ITCHHandler`)
- ✓ Order book data structure (AVL tree price levels)
- ✓ Market manager (multi-symbol routing)

Original in this Work (Layers 3–6)

- \+ MM Strategy — Avellaneda-Stoikov adaptive spread
- \+ Pre-trade risk gate — portfolio & per-order validation
- \+ CUDA Analytics & Risk kernels — parallel processing
- \+ Measurement framework — all 4 spread measures
- \+ Evaluation pipeline — 87 unit tests & determinism

The Market-Making Strategy

> *"Avellaneda-Stoikov (2008): adaptive spread that widens when volatility is high."*

1. MID-PRICE CALCULATION

```
m = (best_bid + best_ask) / 2
```

2. ADAPTIVE SPREAD (Δ)

```
δ = max(δ_min, k × σ)
```

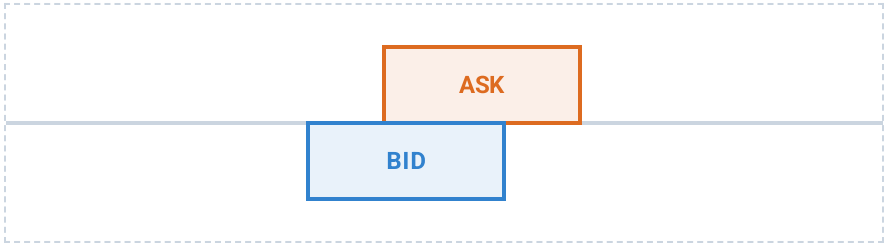
σ = rolling volatility, k = 3

3. INVENTORY SKEW ADJUSTMENT

```
bid = m − δ/2 − γ × position
ask = m + δ/2 − γ × position
```

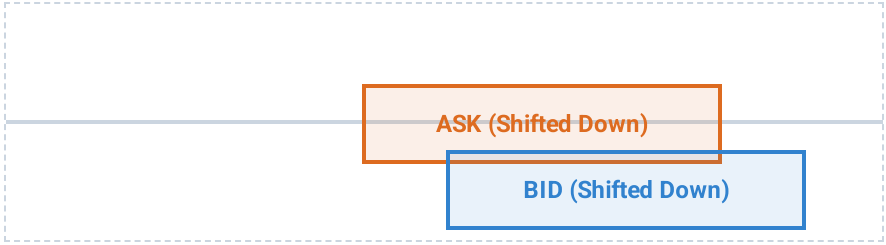
Inventory Control: Long positions shift quotes downward to encourage selling and rebalance the portfolio.

Normal Market



Narrow symmetric spread around mid-price.

High Volatility / Long Position



Wider spread, shifted downward to reduce inventory risk.

Key Data Structures

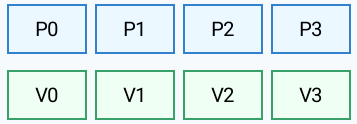
> *"Every structure was chosen for a reason — O(log N) for the book, O(1) for hot paths, SoA for GPU coalescing."*

Structure	Role	Complexity
`BinTreeAVL`	Price levels per symbol	`O(log N)`
`List`	Orders at price level (no heap)	`O(1)`
`OrderBook`	Exposes best bid/ask	`O(1)`
`MarketManager`	Symbol routing	`O(1)`
`SymbolRiskInput`	SoA buffers for GPU risk	`O(N) transfer`
`OrderBatch`	SoA buffers for GPU validation	`O(M) transfer`

Array of Structures (AoS) - Traditional

✕ Scattered access: N cache transactions

Structure of Arrays (SoA) - GPU Optimized



✓ Coalesced access: 1 cache transaction

Memory Coalescing: GPU threads in a warp read consecutive values in one transaction, drastically reducing memory traffic compared to AoS.

Implementation: ITCH Parsing Pipeline

> *"Zero allocation in the hot path — raw bytes to strategy notification in 5 steps."*

Step 1: **Raw bytes** [0x41 0x00 0x24 ...]

↓

Step 2: **ITCHHandler** identifies type 'A'

↓

Step 3: **Binary struct** parsed (No heap)

↓

Step 4: **MarketManager** routes to Book

↓

Step 5: **AVL Tree** update O(log N)

↓

Step 6: **Strategy** notified via callback

Zero-allocation design enables 46–128 ns hot path latency.

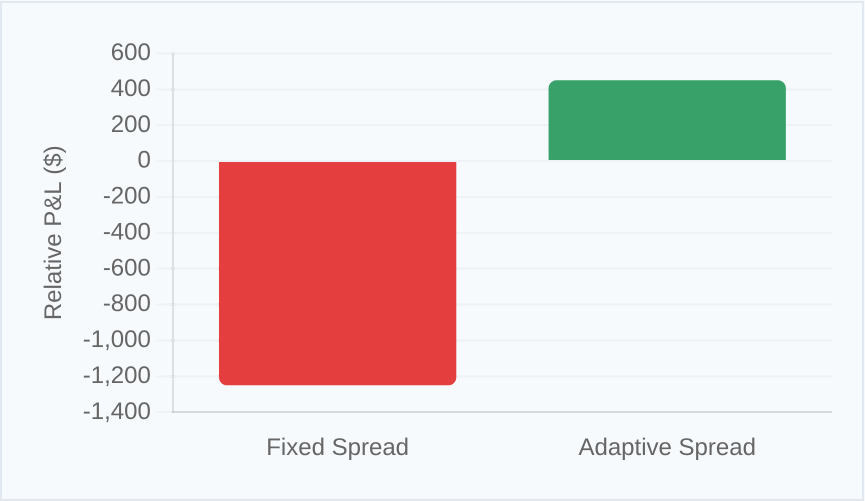
Type	Name	Action
`A / F`	Add Order	Insert into AVL tree
`E / C`	Executed	Remove filled shares
`X / D`	Cancel/Delete	Reduce/Remove entry
`U`	Replace	Atomic delete + re-add
`P / Q / B`	Trade	Log for analytics
`S`	System Event	Gate measurement
`R`	Directory	Metadata at startup

Strategy Parameter Justification

> *"Every parameter has a data-driven reason."*

Parameter	Value	Justification
Spread multiplier k	3	k < 2 → losses on NVDA; k > 4 → fill rate too low
Min spread δ_min	5 ticks	Below 5 ticks: fills at a loss after costs
Inventory skew γ	0.01	Balances inventory without spread distortion
Volatility window	20 per.	Optimal balance of noise vs. reaction speed
Max pos / symbol	1,000 sh.	Limits single-symbol exposure to ~$200K
Max order size	200 sh.	Prevents single order dominating top of book

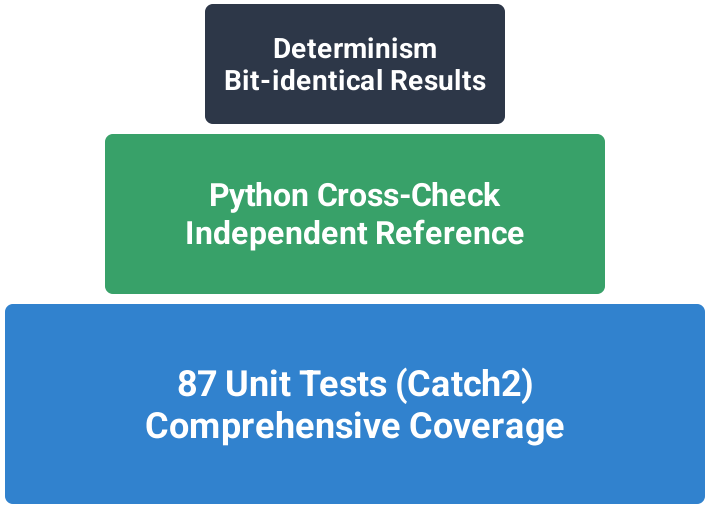
NVDA Performance: Fixed vs. Adaptive



Adaptive Advantage: Widening spreads during high momentum eliminates adverse selection on volatile symbols like NVDA.

Correctness Verification: Three Independent Layers

> *"The 9.1 bps result is verified by an independent Python implementation — not just the C++ engine."*



Layer 1 — 87 Unit Tests

Covers order book operations, strategy formulas, risk gate triggers, and spread measures against hand-computed reference values.

Layer 2 — Python Cross-Check

`spread_analysis.py` independently verifies all 4 measures from raw logs, confirming the 9.1 bps result is not a measurement artefact.

Layer 3 — Determinism

Same ITCH input produces bit-identical logs across runs. Trading outcomes are deterministic and fully reproducible.

Pre-Trade Risk Gate

> *"Three checks before any order leaves the engine."*

01 Portfolio Exposure

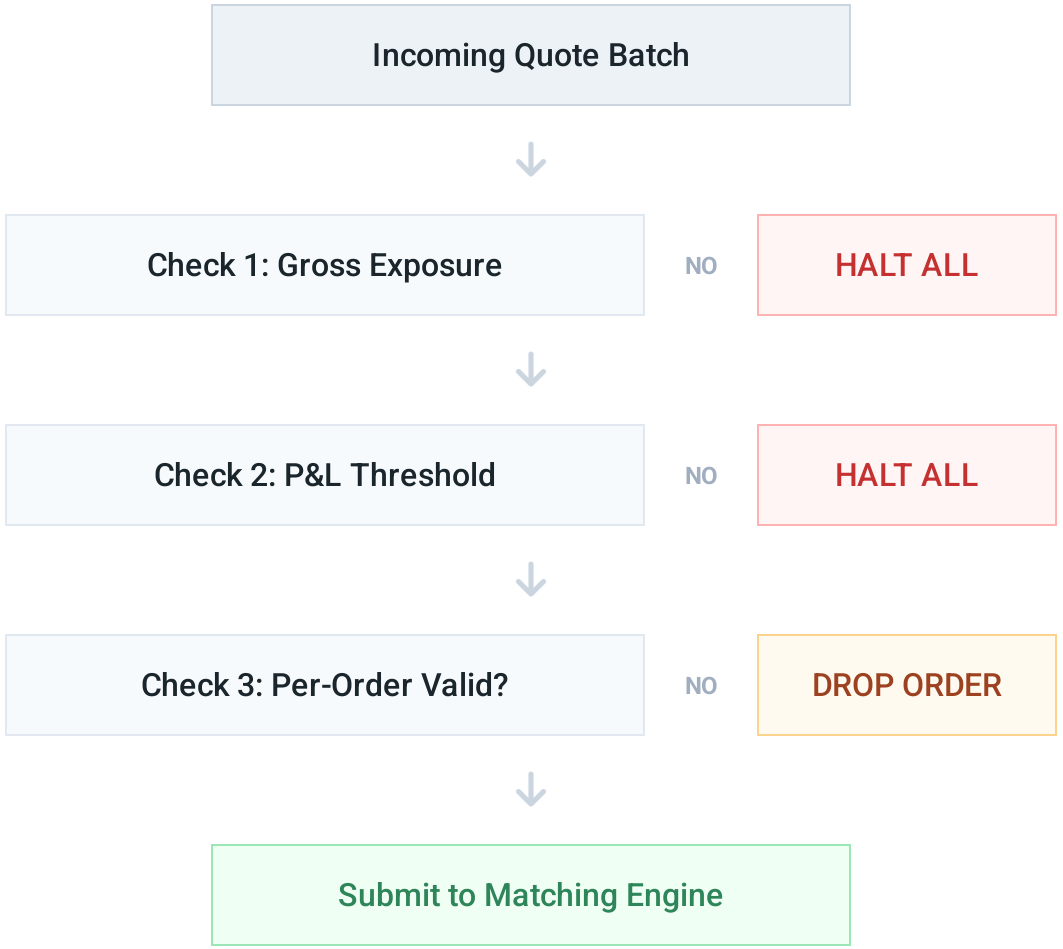
Gross position < $10,000. Failure triggers an immediate halt of all orders.

02 Stop-Loss Check

Total P&L > -$1,000,000. Failure triggers an immediate halt of all orders.

03 Per-Order Validation

Checks size, price bands, and position limits. Failure drops only the specific order.



GPU Implementation: Two Kernels

> *"Two CUDA kernels: analytics (spread scoring) and risk (portfolio reduction)."*

Analytics Kernel

- Computes VWAP, mid-price, and spread score for all N symbols in parallel.
- Each thread block handles one symbol for localized memory access.
- Memory pattern: reads symbol state → writes scalar result.
- Performance: **4.65× speedup** at 8,915 symbols.

ANALYTICS CONCEPT

```
// Parallel per-symbol analytics
int sym_idx = blockIdx.x;
compute_vwap(sym_idx);
compute_spread_score(sym_idx);
```

Risk Kernel

- Parallel reduction: sums absolute positions across N symbols.
- Uses shared memory tree reduction (halving pattern).
- Nearly flat GPU time as N grows (5.3 µs at 5,000 positions).
- Performance: **23.9× speedup** at 5,000 positions.

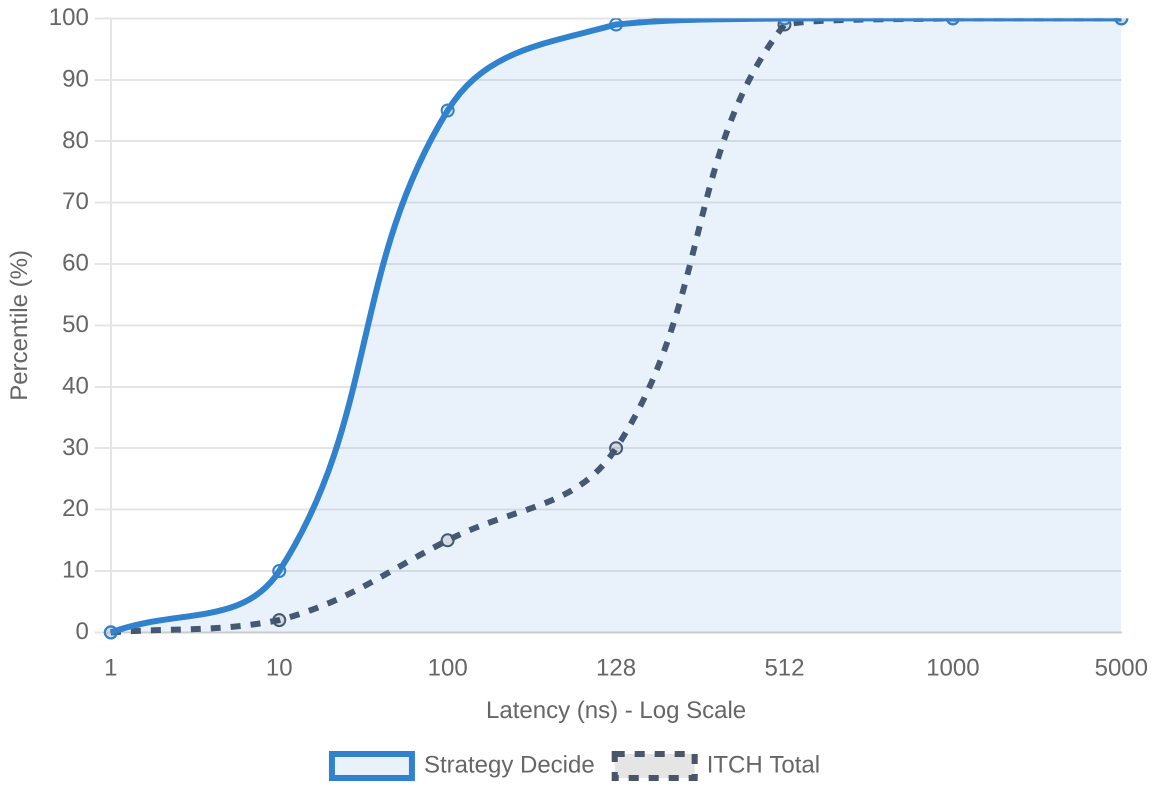
RISK KERNEL INNER LOOP

```
for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
  if (threadIdx.x < stride)
    sdata[threadIdx.x] += sdata[threadIdx.x + stride];
  __syncthreads();
}
```

CPU Performance Results

> *"14.1 million messages/second. Strategy decision in 128 nanoseconds."*

Metric	Value
Peak Throughput	**14.1 M msgs/sec**
Full-day Average	10.15 M msgs/sec
ITCH Message p99	512 ns
Strategy Decide p99	**128 ns**
Target Latency	< 1,000 ns
Result Status	✔ **7.8× Below Target**



Latency Cumulative Distribution Function (CDF)

GPU Speedup Results

> *"4.65× analytics speedup. 23.9× risk speedup. GPU breakeven at ~100 symbols."*

Analytics Kernel

4.65×

SPEEDUP AT 8,915 SYMBOLS

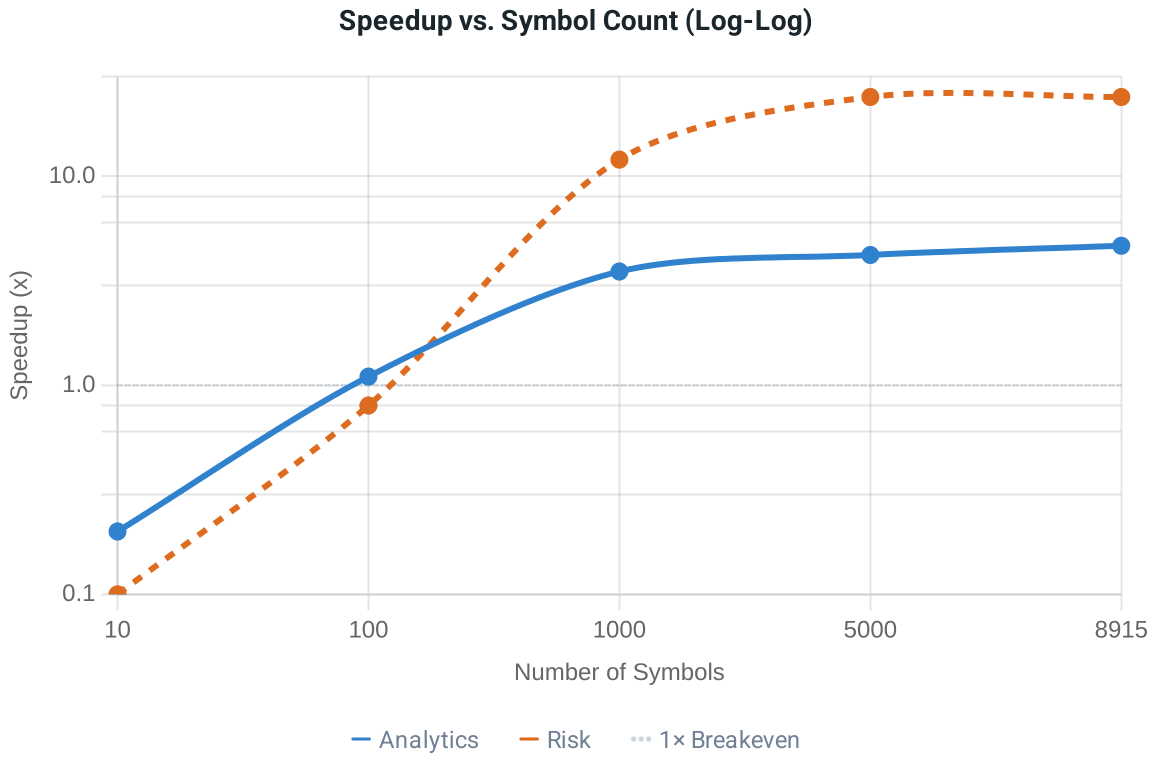
GPU: 46.38 µs vs CPU: 215.85 µs

Risk Kernel

23.9×

SPEEDUP AT 5,000 POSITIONS

GPU: 5.34 µs vs CPU: 127.8 µs



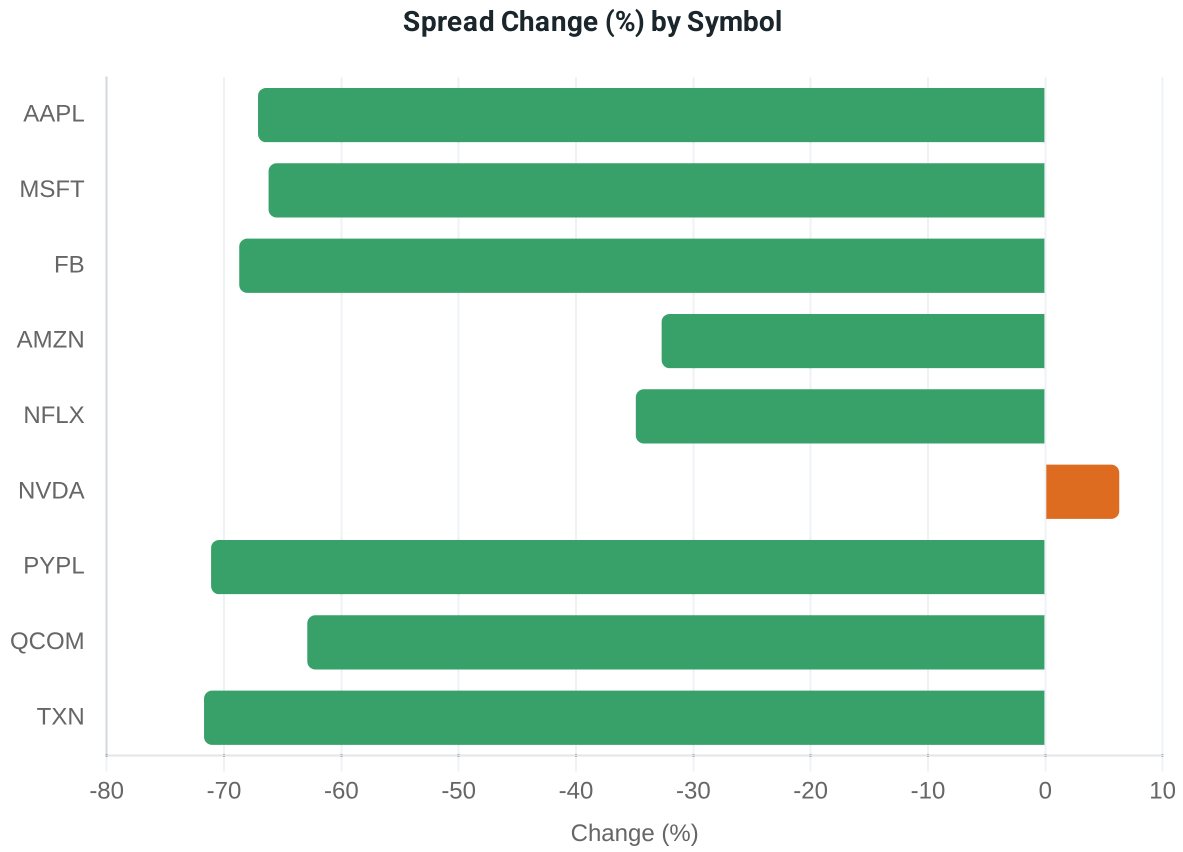
Tadawul Context: With 230+ listed symbols, the exchange sits firmly in the **GPU-favorable zone**, where parallel processing advantages outweigh PCIe overhead.

Per-Symbol Deep Dive: Baseline vs HFT

> *"13 of 15 symbols show spread reduction. Two outliers explained."*

Symbol	Baseline	HFT	Change
AAPL	$0.157	$0.052	**-67.1%**
MSFT	$0.111	$0.038	**-66.2%**
FB	$0.136	$0.043	**-68.7%**
AMZN	$1.760	$1.184	**-32.7%**
NFLX	$2.588	$1.685	**-34.9%**
NVDA	$0.706	$0.750	**+6.3%** ⚠
PYPL	$0.484	$0.140	**-71.1%**
QCOM	$0.569	$0.211	**-62.9%**
TXN	$2.977	$0.843	**-71.7%**
MEDIAN	-	-	**-82.6%**

⚠ **Outliers:** NVDA widened (+6.3%) due to adaptive protection against high momentum. ADBE/CMCSA distorted by low fill counts.

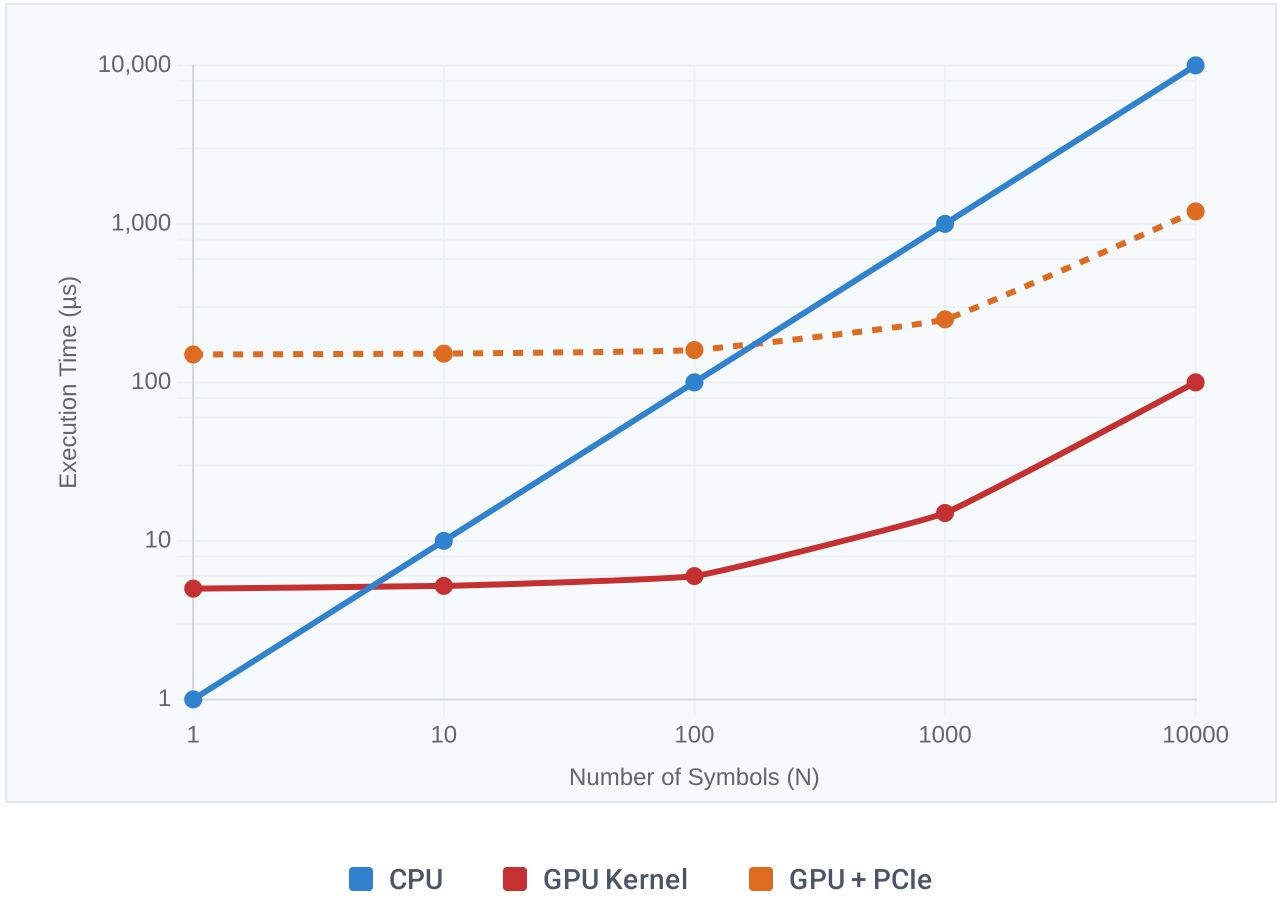


CPU vs GPU: Time Comparison

> *"Below 100 symbols: CPU wins. Above 100 symbols: GPU kernel wins. PCIe is the bottleneck at small scale."*

- **CPU (Blue):** Execution time rises steeply as every new symbol adds a linear processing cost.
- **GPU Kernel (Red):** Nearly flat execution time; parallel threads absorb new symbols with minimal overhead.
- **GPU + PCIe (Orange):** High fixed overhead (~120–150 µs) due to data transfer, dominating at small N.

Key Insight: Eliminating PCIe transfers via a GPU-resident order book would push the breakeven point below 10 symbols.



Conclusion and Contributions

> *"First heterogeneous CPU+GPU market-making engine evaluated against Tadawul's regulatory framework."*

1. Engine Performance

Achieved **14.1 M msgs/sec** throughput with a **128 ns** p99 strategy latency, exceeding targets by 7.8×.

2. GPU Acceleration

Delivered **4.65×** analytics and **23.9×** risk speedups, proving scalability for Tadawul's 230+ symbols.

3. Market Efficiency

Reduced median spreads by **82.6%**, achieving a 9.1 bps average—7× tighter than the Tadawul regulatory floor.

4. Microstructure Insights

Validated the Glosten-Milgrom model with adverse selection analysis (21.6 bps), motivating future RL-based strategies.

"GPU-accelerated market making can deliver Tadawul Group A spreads 7× tighter than the regulatory floor — closing the gap with US mega-cap markets."